Question 2

(a) Value Iteration Function

The code computes the approximately optimal action-value function $Q^*(s, a)$using the value iteration algorithm. It iteratively updates Q-values based on the Bellman optimality equation, where each Q-value is calculated as the sum of immediate reward and discounted expected future value. The transition probabilities $P(s' \mid s, a)$are obtained from the kernel using the indexing row_idx $= s \times A + a$, which maps each (state, action) pair to the correct row in the kernel. For each iteration, the maximum Q-value over all actions for each next state is used to compute the optimal future return. The process continues until the maximum change in Q-values across all states and actions becomes smaller than a predefined threshold ϵ, ensuring convergence. This method directly computes the optimal Q-function without explicitly computing the policy.

(b) Induced Policy Function

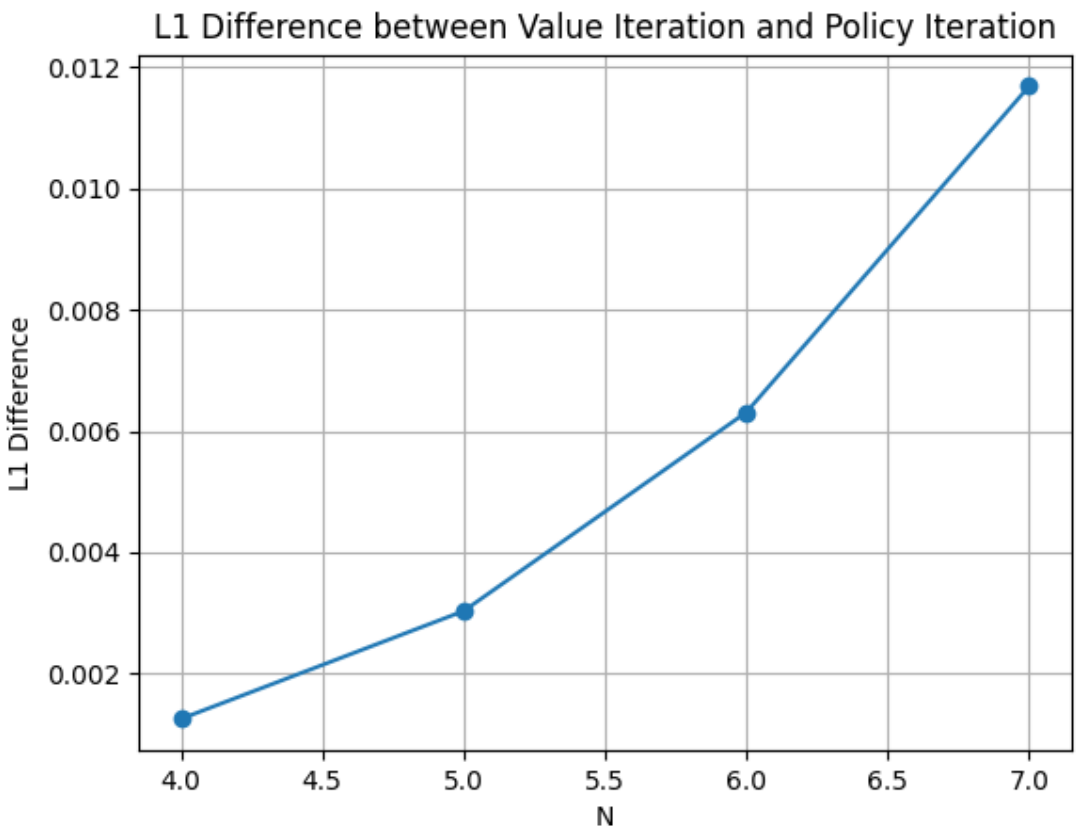
The code extracts a policy $\pi(s)$from a given Q-function by selecting actions that maximize the Q-value in each state. For every state, the action(s) with the highest Q-value are identified, and probabilities are assigned accordingly. If there is a single optimal action, it is assigned probability 1, while if multiple actions share the maximum value, the probability is distributed equally among them. This results in a greedy policy with respect to the Q-function. The function ensures that each row of the policy matrix forms a valid probability distribution over actions.

(c) Policy Iteration Function

The code computes the optimal Q-function using the policy iteration algorithm, which alternates between policy evaluation and policy improvement. Initially, a random policy is assumed. In the policy evaluation step, the state value function $V^\pi(s)$is computed for the current policy using iterative updates. In the policy improvement step, a new policy is generated by selecting actions that maximize the expected return, using the relation $Q(s, a) = R(s, a) + \gamma \sum_{s'} P(s' \mid s, a)V(s')$. The indexing row_idx $= s \times A + a$is again used to extract transition probabilities from the kernel. This process repeats until the policy converges, i.e., no further changes occur between successive iterations. The final Q-function corresponds to the optimal policy.

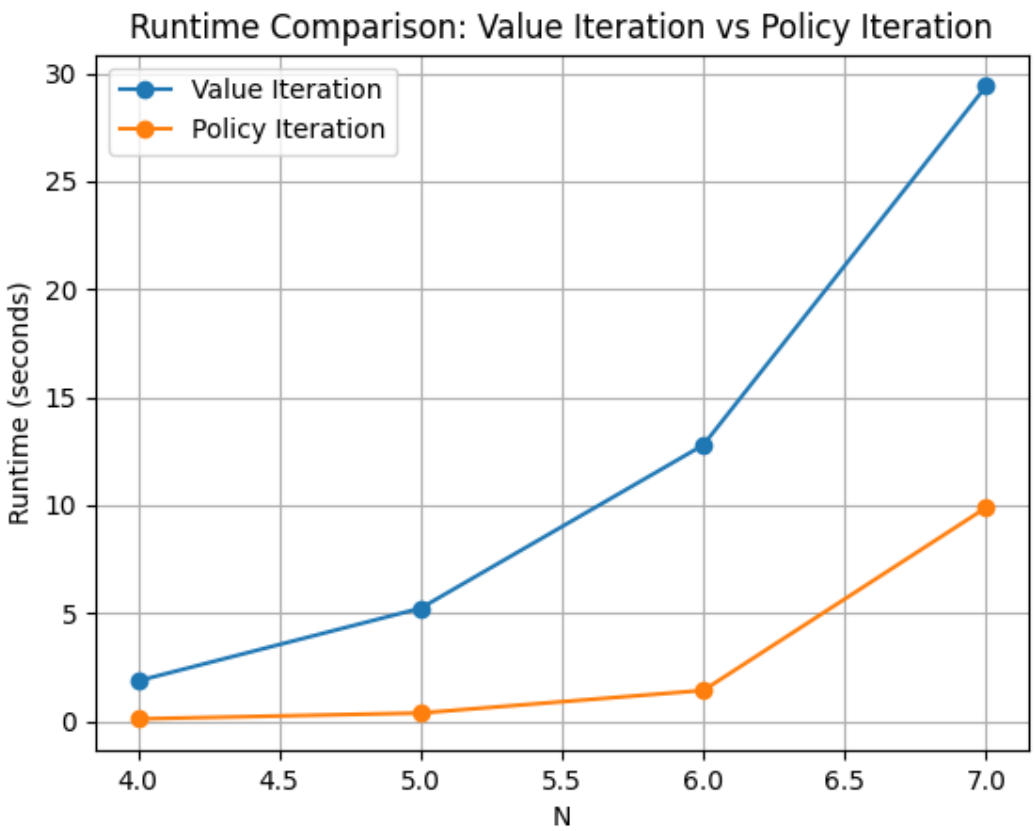
(d) L1 Difference Plot

In this part, both value iteration and policy iteration are executed for different grid sizes N, and their resulting Q-functions are compared. The difference between the two Q-functions is measured using the L1 norm, defined as the sum of absolute differences between corresponding Q-values. This provides a quantitative measure of how close the two methods are in terms of their computed solutions. Since both algorithms are expected to converge to the same optimal solution, the L1 difference should ideally be close to zero. The results are plotted as a function of N, showing how the difference behaves as the problem size increases.



(e) Runtime Analysis Plot

This part evaluates the computational efficiency of value iteration and policy iteration by measuring their execution times for different values of N. For each grid size, the total runtime required to compute the Q-function is recorded using time measurements before and after execution. Since the number of states grows as N^4, both algorithms become computationally expensive as Nincreases. The recorded runtimes are plotted against N, allowing a comparison of how the two methods scale with problem size. Typically, policy iteration converges in fewer iterations but involves more computation per iteration, while value iteration requires more iterations but each step is simpler



Naming Conventions and Implementation Details

A consistent and structured naming convention has been followed throughout the implementation to ensure clarity, readability, and direct correspondence with standard reinforcement learning notation.

Naming Conventions

All files follow the format:
Q<question_number><part>_<description>.py
This clearly indicates which part of the assignment the file belongs to and its functionality. For example:

- Q2a_value_iteration_func.py → Value Iteration implementation
- Q2b_induced_policy_func.py → Policy extraction from Q-function
- Q2c_policy_iteration_func.py → Policy Iteration implementation

Function names follow **snake_case** and are descriptive of their purpose:

- value_iteration → computes optimal Q-function
- induced_policy → extracts greedy policy from Q
- policy_iteration → computes optimal Q using policy iteration

Variable Naming

Variable names are chosen to closely match mathematical notation:

- **S** → Number of states
- **A** → Number of actions
- **s_idx** → Index of current state
- **a_idx** → Index of current action
- **row_idx = s * A + a** → Maps (state, action) pair to a unique row in the kernel
- **P_sa** → Transition probability vector $P(s' \mid s, a)$
- **V** → State value function $V(s)$
- **Q** → Action value function $Q(s, a)$

This mapping ensures that the implementation directly reflects the mathematical equations used in the algorithms.

Implementation Details

The implementation is modular, with each function written in a separate file as required. Value iteration directly updates the Q-function using the Bellman optimality equation and checks convergence using a threshold ϵ. Policy iteration alternates between evaluating the current policy and improving it using the computed value function, repeating until convergence. The induced policy function extracts a greedy policy by selecting actions with maximum Q-values.

The transition kernel is accessed using the indexing scheme row_idx $= s \times A + a$, which efficiently flattens the (state, action) pair into a single dimension. Matrix operations such as dot products are used to compute expected future values, ensuring computational efficiency.